

Advanced Statistics - Assignment 2

Daniele Ghezzi - 2195461
Physics of Data - University of Padua

May 26, 2026

You are given a dataset of integer-valued observations

$$D = \{x_1, x_2, \dots, x_N\}$$

contained in the file `assignment2_changepoint_counts.csv`. The data represent the number of events observed in identical consecutive time intervals. In this assignment, the order of the observations matters. The goal is to analyze the data using Markov chain Monte Carlo methods. All MCMC algorithms must be coded directly in R. Do not use JAGS, Stan, NIMBLE, brms, rstanarm, or other automatic Bayesian sampling software that have not yet been covered in the lectures. Throughout the assignment, use the Gamma density parameterized by shape and rate. Unless otherwise stated, use

$$p(\lambda|\alpha, \beta) = \frac{\beta^\alpha}{\Gamma(\alpha)} \lambda^{\alpha-1} e^{-\beta\lambda}, \quad \lambda > 0$$

with $\alpha = 1$ and $\beta = 0.1$.

1 Bayesian change-point model

Assume that the data are generated from a Poisson model with one unknown change point. Before the change point, the event rate is λ_1 ; after the change point, the event rate is λ_2 . More precisely,

$$\begin{aligned} x_i | \lambda_1, m &\sim \text{Poisson}(\lambda_1), & i = 1, \dots, m \\ x_i | \lambda_2, m &\sim \text{Poisson}(\lambda_2), & i = m + 1, \dots, N \end{aligned}$$

where $m \in \mathcal{M} = \{n_{\min}, n_{\min} + 1, \dots, N - n_{\min}\}$, with $n_{\min} = 5$. Take the prior distributions as:

$$\begin{aligned} \lambda_1 &\sim \text{Gamma}(\alpha, \beta) \\ \lambda_2 &\sim \text{Gamma}(\alpha, \beta) \\ m &\sim \text{Unif}(\mathcal{M}) \end{aligned}$$

Before going deeper into the assignment, we evaluate some general statistics in R.

```
1 # load data
2 filepath <- '/home/dghezzi/AdvancedStatistics/ex2/assignment2_changepoint_counts.csv'
3 data <- read.csv(filepath)
4 set.seed(4) # for reproducibility
5
6 # summary statistics
7 summary(data$x)
8 sprintf("Variance: %.2f", var(data$x))
9 sprintf("N = %d", length(data$x))
10
11 alpha_prior <- 1
12 beta_prior <- 0.1
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
0.00	3.00	4.00	4.05	5.00	9.00

[1] "Variance: 3.86"
[1] "N = 120"

We notice that the mean and the variance are quite similar.

1.1

Write the likelihood $p(D|\lambda_1, \lambda_2, m)$.

The likelihood is given by the product of the individual Poisson probability mass functions. Assuming the observations are conditionally independent given m , λ_1 , and λ_2 , the rate is λ_1 for the first m observations ($1 \leq i \leq m$) and λ_2 for the remaining observations ($m+1 \leq i \leq N$). Therefore:

$$\begin{aligned}
p(D|\lambda_1, \lambda_2, m) &= \prod_{i=1}^m p(x_i|\lambda_1) \prod_{i=m+1}^N p(x_i|\lambda_2) \\
&= \prod_{i=1}^m \left(\frac{\lambda_1^{x_i} e^{-\lambda_1}}{x_i!} \right) \prod_{i=m+1}^N \left(\frac{\lambda_2^{x_i} e^{-\lambda_2}}{x_i!} \right) \\
&= \left(\prod_{i=1}^m \frac{1}{x_i!} \right) \lambda_1^{\sum_{i=1}^m x_i} e^{-m\lambda_1} \lambda_2^{\sum_{i=m+1}^N x_i} e^{-(N-m)\lambda_2}
\end{aligned} \tag{1}$$

1.2

Write the posterior distribution $p(\lambda_1, \lambda_2, m|D)$ up to a proportionality constant.

We apply Bayes' theorem and neglect the evidence term, as it acts as a normalizing constant:

$$p(\lambda_1, \lambda_2, m|D) \propto p(D|\lambda_1, \lambda_2, m)p(\lambda_1)p(\lambda_2)p(m) \tag{2}$$

The priors for the rates are Gamma distributions, while the prior for m is a discrete uniform distribution, which we can treat as a constant and absorb into the proportionality factor. Finally, by multiplying the likelihood by the priors and dropping all constant terms (such as the factorial terms from the likelihood and the denominators from the Gamma priors), the posterior distribution is:

$$\begin{aligned}
p(\lambda_1, \lambda_2, m|D) &\propto \left(\lambda_1^{\sum_{i=1}^m x_i} e^{-m\lambda_1} \lambda_2^{\sum_{i=m+1}^N x_i} e^{-(N-m)\lambda_2} \right) \left(\lambda_1^{\alpha-1} e^{-\beta\lambda_1} \right) \left(\lambda_2^{\alpha-1} e^{-\beta\lambda_2} \right) \\
&= \lambda_1^{\alpha-1+\sum_{i=1}^m x_i} e^{-(\beta+m)\lambda_1} \lambda_2^{\alpha-1+\sum_{i=m+1}^N x_i} e^{-(\beta+N-m)\lambda_2}
\end{aligned} \tag{3}$$

1.3

Define the cumulative sums $S_m = \sum_{i=1}^m x_i$ and $S_N = \sum_{i=1}^N x_i$. Show that, conditionally on m , the posterior distributions of the two rates are Gamma distributions. Give their parameters.

We notice that, if we condition on m , the conditional posterior distribution factors into the product of two independent Gamma distributions corresponding to the rate parameters:

$$\begin{aligned}
p(\lambda_1, \lambda_2|m, D) &= p(\lambda_1|m, D)p(\lambda_2|m, D) \\
&\propto \left(\lambda_1^{\alpha+S_m-1} e^{-(\beta+m)\lambda_1} \right) \left(\lambda_2^{\alpha+S_N-S_m-1} e^{-(\beta+N-m)\lambda_2} \right)
\end{aligned} \tag{4}$$

Therefore, for $\lambda_1 \sim \text{Gamma}(\alpha_{\lambda_1}, \beta_{\lambda_1})$, we recognize the parameters as:

$$\alpha_{\lambda_1} = \alpha + S_m \quad \text{and} \quad \beta_{\lambda_1} = \beta + m$$

Similarly, for $\lambda_2 \sim \text{Gamma}(\alpha_{\lambda_2}, \beta_{\lambda_2})$, we recognize the parameters as:

$$\alpha_{\lambda_2} = \alpha + S_N - S_m \quad \text{and} \quad \beta_{\lambda_2} = \beta + N - m$$

1.4

Derive the conditional posterior distribution $p(m|\lambda_1, \lambda_2, D)$. Explain why it is a discrete distribution over $\mathcal{M}$.

If we condition on λ_1 and λ_2 and isolate the terms that depend only on m , we find that, up to a proportionality constant, the conditional posterior distribution for m is:

$$\begin{aligned} p(m|\lambda_1, \lambda_2, D) &\propto p(D|\lambda_1, \lambda_2, m)p(m) \\ &\propto \left(\lambda_1^{S_m} e^{-m\lambda_1}\right) \left(\lambda_2^{S_N - S_m} e^{-(N-m)\lambda_2}\right) \\ &\propto \lambda_1^{S_m} e^{-m\lambda_1} \lambda_2^{-S_m} e^{m\lambda_2} \\ &= \left(\frac{\lambda_1}{\lambda_2}\right)^{S_m} e^{-m(\lambda_1 - \lambda_2)} \end{aligned} \tag{5}$$

If we normalize over all possible change points in the set $\mathcal{M}$:

$$p(m|\lambda_1, \lambda_2, D) = \frac{\left(\frac{\lambda_1}{\lambda_2}\right)^{S_m} e^{-m(\lambda_1 - \lambda_2)}}{\sum_{k \in \mathcal{M}} \left(\frac{\lambda_1}{\lambda_2}\right)^{S_k} e^{-k(\lambda_1 - \lambda_2)}} \tag{6}$$

This is a discrete probability mass function because $m \in \mathcal{M}$, which is a finite set of discrete time indices.

1.5

Explain why this model is a natural case for MCMC, even though some conditional distributions are available analytically.

This model is a natural candidate for MCMC methods and for Gibbs sampling in particular. It combines continuous parameters with a discrete parameter, and computing the exact evidence is analytically complex. However, as derived in the previous steps, the full conditional distributions for all parameters are available analytically (Gamma and m pmf). Gibbs sampling specifically exploits these conditional distributions to iteratively draw samples from the joint posterior, even if it is not normalized, bypassing the need to compute the evidence.

2 Gibbs sampling

Implement a Gibbs sampler for $(\lambda_1, \lambda_2, m)$.

2.1

Write explicitly the three update steps of one Gibbs iteration. In the update of m , compute the probabilities on the log scale and normalize them carefully.

The function `gibbs_sampling_posterior` implements the Gibbs sampling algorithm to draw samples from the joint posterior distribution. The procedure can be divided into three main steps:

1. **Initialization:** The algorithm initializes the parameter vectors with the provided starting values.
2. **λ_1 and λ_2 updates:** At each iteration t , new values for the rates are drawn from their respective full conditional Gamma distributions. The shape and rate parameters are updated at each step because the cumulative sums changes due to the most recently sampled change point $m^{(t-1)}$.
3. **m update:** The algorithm evaluates the unnormalized log-posterior probability for every value in $\mathcal{M}$, conditioning on the newly sampled rates $\lambda_1^{(t)}$ and $\lambda_2^{(t)}$. To prevent numerical underflow when exponentiating these probabilities, we then subtract the maximum log-probability before exponentiation. The resulting weights are then normalized to form a valid probability mass function, from which the new change point $m^{(t)}$ is sampled.

```

1 # 2.a write the three update steps of one Gibbs iteration
2 gibbs_sampling_posterior <- function(n_steps, data, alpha, beta, start = c(1, 1, 10)) {
3
4   # initialize empty vectors
5   lambda1 <- numeric(n_steps)
6   lambda2 <- numeric(n_steps)
7   m <- numeric(n_steps)
8
9   # initialize first values
10  lambda1[1] <- start[1]
11  lambda2[1] <- start[2]
12  m[1] <- start[3]
13
14  sumN <- sum(data$x)
15  N <- length(data$x)
16  Mset <- seq(5, N-5, 1)
17
18  for (t in 2:n_steps) {
19    current_m <- m[t-1]
20    sumM_current <- sum(data$x[1:current_m])
21
22    # lambda updates
23    lambda1[t] <- rgamma(1, shape = alpha + sumM_current, rate = beta + current_m)
24    lambda2[t] <- rgamma(1, shape = alpha + sumN - sumM_current, rate = beta + N - current_m
25  )
26
27    # m update
28    logm <- numeric(length(Mset))
29
30    for (i in 1:length(Mset)) {
31      m_cand <- Mset[i]
32      sumM_cand <- sum(data$x[1:m_cand])
33
34      logm[i] <- sumM_cand * log(lambda1[t]) - m_cand * lambda1[t] + (sumN - sumM_cand) *
35      log(lambda2[t]) - (N - m_cand) * lambda2[t]
36    }
37
38    weights <- exp(logm - max(logm))
39    probs_m <- weights / sum(weights)
40
41    m[t] <- sample(x = Mset, size = 1, prob = probs_m)
42  }
43
44  print("Gibbs sampling completed.")
45  return(cbind(lambda1 = lambda1, lambda2 = lambda2, m = m))
46 }

```

2.2

Run at least three chains, starting from different initial values of m , λ_1 and λ_2 . Produce trace plots for λ_1 , λ_2 , and m .

```
1 # 2.b Run three chains starting from different values and show the trace plots
2 starts <- list(
3   c(4, 5, 10),
4   c(6, 4, 20),
5   c(2, 6, 25)
6 )
7
8 traces <- vector("list", length(starts))
9 n_steps <- 50000
10
11 for (i in 1:length(starts)) {
12   traces[[i]] <- gibbs_sampling_posterior(n_steps = n_steps,
13     data = data,
14     alpha = alpha_prior,
15     beta = beta_prior,
16     start = starts[[i]])
17 }
18
19 par(mfrow = c(3, 3), mar = c(2.5, 2.5, 2, 1), mgp = c(1.5, 0.5, 0))
20 chain_cols <- c("blue", "red", "forestgreen")
21 param_names <- c("lambda1", "lambda2", "m")
22 param_exprs <- c(expression(lambda[1]), expression(lambda[2]), expression(m))
23
24 for (p in 1:3) {
25
26   param <- param_names[p]
27   y_label <- param_exprs[[p]]
28
29   global_ylim <- range(c(traces[[1]][, param], traces[[2]][, param], traces[[3]][, param]))
30
31   for (i in 1:3) {
32
33     plot_title <- paste(param_names[p], "- Chain", i)
34
35     plot(traces[[i]][, param], type = "l", col = chain_cols[i],
36       ylim = global_ylim, main = plot_title,
37       xlab = "Iteration", ylab = y_label,
38       cex.main = 1.2, cex.lab = 1)
39   }
40 }
41 par(mfrow = c(1, 1))
```

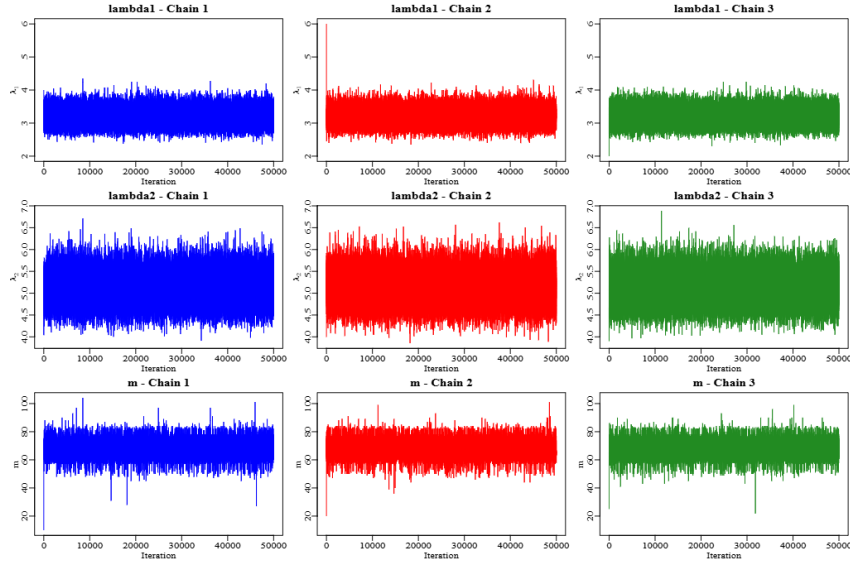


Figure 1: Trace plots for three different Markov chains. We can observe the initial burn-in period, during which the chains converge from their respective starting points and stabilize.

2.3

Decide on an appropriate burn-in period and justify your choice.

To determine the burn-in period we rely on the trace plots produced in the previous step, as they show the point at which the chains forget their initial starting values and converge to the stationary distribution. Observing Figure 1, the chains appear to converge very rapidly. However, to be conservative and ensure our samples are drawn from the posterior distribution, we choose a burn-in period of 5000 iterations. We discard these initial samples from all chains. After discarding the burn-in phase, we plot the empirical marginal posterior distributions for the three parameters using the remaining samples.

```

1 # 2.c burn-in period?
2 burn_in <- 5000
3 sample_gibbs <- vector("list", length(traces))
4 for (i in 1:length(traces)) {
5   sample_gibbs[[i]] <- traces[[i]][(burn_in + 1):n_steps, ]
6 }
7
8 # plot histogram of the samples
9 lambda1_samples <- sample_gibbs[[1]][, "lambda1"]
10 lambda2_samples <- sample_gibbs[[1]][, "lambda2"]
11 m_samples <- sample_gibbs[[1]][, "m"]
12 par(mfrow = c(1, 3), mar = c(2.5, 2.5, 3.5, 1))
13 hist(lambda1_samples, breaks = 30, main = expression(lambda[1]), xlab = expression(lambda[1]),
14       col = "lightblue", border = "white")
15 hist(lambda2_samples, breaks = 30, main = expression(lambda[2]), xlab = expression(lambda[2]),
16       col = "lightblue", border = "white")
17 hist(m_samples, breaks = 30, main = "m", xlab = "m", col = "lightblue", border = "white")
18 par(mfrow = c(1, 1))

```

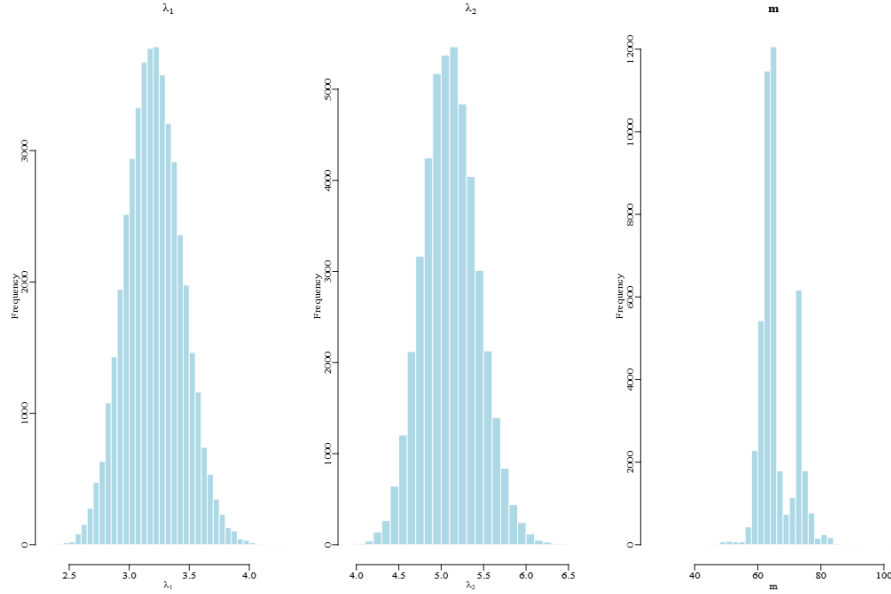


Figure 2: Empirical marginal posterior distributions for λ_1 , λ_2 , and m , generated from the first Markov chain after discarding a 5000 step burn-in period.

2.4

Plot the autocorrelation function for λ_1 , λ_2 , and m . Estimate the effective sample size for λ_1 , λ_2 , and m .

We compute the autocorrelation function (ACF) for the posterior samples of each parameter up to a maximum lag of 100. We calculate the integrated autocorrelation time τ_{int} by summing the autocorrelations ρ_k over contiguous positive lags, truncating the sum as soon as the autocorrelation drops to or below zero to avoid accumulating noise from higher lags. The estimator is given by

$$\tau_{\text{int}} = 1 + 2 \sum_k \rho_k$$

Therefore, we can estimate the Effective Sample Size (ESS) using:

$$\text{ESS} = \frac{N}{\tau_{\text{int}}} \quad (7)$$

where N is the total number of post-burn-in samples.¹

```
1 # 2.d Autocorrelation? Effective sample size?
2 lagMAX <- 100
3 par(mfrow = c(1, 3), mar = c(2.5, 2.5, 3.5, 1))
4 acf_lambda1 <- acf(lambda1_samples, lag.max = lagMAX, main = "ACF for lambda1")
5 acf_lambda2 <- acf(lambda2_samples, lag.max = lagMAX, main = "ACF for lambda2")
6 acf_m <- acf(m_samples, lag.max = lagMAX, main = "ACF for m")
7 par(mfrow = c(1, 1))
8
9 calculate_tau_int <- function(acf) {
10
11   lags <- as.numeric(acf$lag)
12   rho <- as.numeric(acf$acf)
13   positive_lags <- which(rho[-1] > 0)
14
15   if (length(positive_lags) == 0) {
```

¹Unlike the formula presented in the lecture notes, we deliberately omit the factor of 2 in the denominator. This convention is indeed widely adopted; see, for instance, the [Stan Reference Manual](#) or [ESS derivation](#).

```

16   return(1)
17 } else {
18   last_positive <- positive_lags[1]
19   for (i in seq_along(positive_lags)) {
20     if (positive_lags[i] == i) {
21       last_positive <- positive_lags[i]
22     } else {
23       break
24     }
25   }
26   return(1 + 2 * sum(rho[2:(last_positive + 1)]))
27 }
28 }
29
30 tau_int_lambda1 <- calculate_tau_int(acf_lambda1)
31 tau_int_lambda2 <- calculate_tau_int(acf_lambda2)
32 tau_int_m <- calculate_tau_int(acf_m)
33
34 N_samples <- length(lambda1_samples)
35 ess_lambda1 <- N_samples / tau_int_lambda1
36 ess_lambda2 <- N_samples / tau_int_lambda2
37 ess_m <- N_samples / tau_int_m
38 print(paste("ESS for lambda1:", round(ess_lambda1, 0)))
39 print(paste("ESS for lambda2:", round(ess_lambda2, 0)))
40 print(paste("ESS for m:", round(ess_m, 0)))

```

```

[1] "ESS for lambda1: 34596"
[1] "ESS for lambda2: 37494"
[1] "ESS for m: 32380"

```

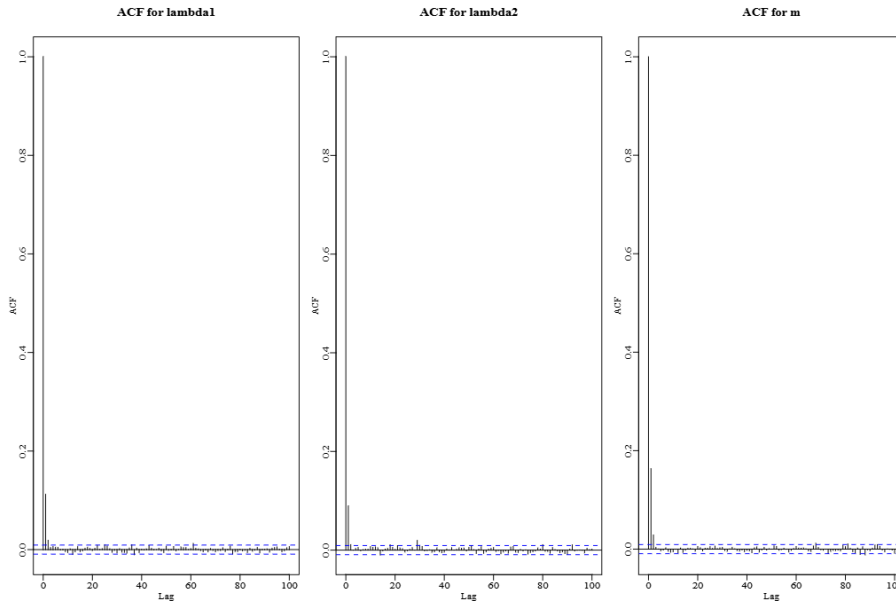


Figure 3: Autocorrelation function (ACF) plots for λ_1 , λ_2 , and m . The plots illustrate how the correlation between successive samples decays as the lag increases. At higher lags the residual autocorrelation fluctuates around zero due to noise.

2.5

Report posterior summaries for λ_1 , λ_2 , and m : posterior mean, posterior median, posterior standard deviation, MAP estimate, and a 95% credible interval.

To evaluate the posterior mean, median, and standard deviation, we simply compute the respective sample statistics over the post burn-in MCMC chains. For the 95% credible interval, rather than using standard central quantiles, we implement a function to compute the Highest Posterior Density (HPD) interval. This is particularly important because the m posterior distribution is multi-modal and the HPD provides the narrowest possible interval containing 95% of the posterior mass. In the code, we do this by sorting the MCMC samples, defining a sliding window containing exactly 95% of the total samples, and iteratively calculating the width of all possible windows to identify and return the boundaries of the narrowest one.

Finally, we use two distinct methods to obtain the Maximum A Posteriori (MAP) estimate:

1. **MAP Samples:** We evaluate the exact unnormalized log-posterior function for every generated triplet $(\lambda_1^{(t)}, \lambda_2^{(t)}, m^{(t)})$ in our chain, and we select the specific sample that maximizes this value.
2. **MAP KDE:** We apply Kernel Density Estimation (KDE) to the continuous samples of λ_1 and λ_2 to approximate their continuous marginal posterior probability density functions, and we identify the peak of these curves. This method is not applied to m , as it is a discrete variable.

Both methods yield very similar results for the continuous rates.

```

1 # 2.e For each par: posterior mean, posterior median, posterior standard deviation, MAP estimate
  , and a 95% credible interval?
2 get_hpd_interval <- function(samples, prob = 0.95) {
3
4   # the HPD interval is the narrowest interval that contains 95% of the posterior mass
5
6   sorted_samples <- sort(samples)
7   n <- length(sorted_samples)
8
9   # number of samples needed to cover the target probability
10  window_size <- round(n * prob)
11
12  # define the start and end indices for all possible sliding windows across the sorted samples
13  start_idx <- 1:(n - window_size)
14  end_idx <- start_idx + window_size
15
16  # calculate the physical width of every single window
17  window_widths <- sorted_samples[end_idx] - sorted_samples[start_idx]
18
19  # find the index of the narrowest window
20  best_idx <- which.min(window_widths)
21
22  # Return the boundaries of that narrowest window
23  return(c(lower = sorted_samples[best_idx], upper = sorted_samples[end_idx[best_idx]]))
24 }
25
26 posterior_summary <- function(samples) {
27   mean_val <- mean(samples)
28   median_val <- median(samples)
29   sd_val <- sd(samples)
30   ci <- get_hpd_interval(samples, prob = 0.95)
31   return(list(mean = mean_val, median = median_val, sd = sd_val, ci = ci))
32 }
33
34 lambda1_summary <- posterior_summary(lambda1_samples)
35 lambda2_summary <- posterior_summary(lambda2_samples)
36 m_summary <- posterior_summary(m_samples)
37
38 print("Lambda1 Summary:")
39 print(lambda1_summary)

```

```

40 print("Lambda2 Summary:")
41 print(lambda2_summary)
42 print("m Summary:")
43 print(m_summary)
44
45 # to get the MAP estimate, there are two options:
46 # 1) we calculate the log-posterior for each sample and we take the one with the highest value;
47 # 2) we use KDE to estimate the density and we find its maximum.
48 # method 1
49 calculate_log_posterior <- function(lambda1, lambda2, m, data, alpha, beta) {
50   N <- length(data$x)
51
52   sumN <- sum(data$x)
53   sumM <- sum(data$x[1:m])
54
55   log_prior_lambda1 <- dgamma(lambda1, shape = alpha, rate = beta, log = TRUE)
56   log_prior_lambda2 <- dgamma(lambda2, shape = alpha, rate = beta, log = TRUE)
57
58   log_likelihood <- sumM * log(lambda1) - m * lambda1 + (sumN - sumM) * log(lambda2) - (N - m) *
     lambda2
59
60   return(log_prior_lambda1 + log_prior_lambda2 + log_likelihood)
61 }
62
63 log_posteriors <- mapply(calculate_log_posterior,
64                           lambda1 = lambda1_samples,
65                           lambda2 = lambda2_samples,
66                           m = m_samples,
67                           MoreArgs = list(data = data, alpha = alpha_prior, beta = beta_prior))
68 max_index <- which.max(log_posteriors)
69 map_lambda1 <- lambda1_samples[max_index]
70 map_lambda2 <- lambda2_samples[max_index]
71 map_m <- m_samples[max_index]
72 sprintf("MAP estimate for lambda1: %.4f", map_lambda1)
73 sprintf("MAP estimate for lambda2: %.4f", map_lambda2)
74 sprintf("MAP estimate for m: %.0f", map_m)
75
76 # method 2 (no KDE on m since it is discrete)
77 kde_lambda1 <- density(lambda1_samples)
78 kde_lambda2 <- density(lambda2_samples)
79 map_lambda1_kde <- kde_lambda1$x[which.max(kde_lambda1$y)]
80 map_lambda2_kde <- kde_lambda2$x[which.max(kde_lambda2$y)]
81 sprintf("MAP estimate for lambda1 (KDE): %.4f", map_lambda1_kde)
82 sprintf("MAP estimate for lambda2 (KDE): %.4f", map_lambda2_kde)
83
84 # plot kde and map estimates with histogram of the samples
85 par(mfrow = c(1, 3), mar = c(2.5, 2.5, 3.5, 1))
86
87 hist(lambda1_samples, breaks = 30, main = expression(lambda[1]), xlab = expression(lambda[1]),
88       col = "lightblue", border = "white", freq = FALSE)
89 lines(kde_lambda1, col = "red", lwd = 2)
90 abline(v = map_lambda1, col = "blue", lwd = 2, lty = 2)
91 abline(v = map_lambda1_kde, col = "#1e9600", lwd = 2, lty = 3)
92 abline(v = lambda1_summary$ci[1], col = "purple", lwd = 2, lty = 4)
93 abline(v = lambda1_summary$ci[2], col = "purple", lwd = 2, lty = 4)
94 legend("topright", legend = c("MAP KDE", "MAP samples", "95% CI (HPD)"), col = c("#1e9600", "
95   blue", "purple"), lwd = 2, lty = c(3, 2, 4), cex = 0.8)
96
97 hist(lambda2_samples, breaks = 30, main = expression(lambda[2]), xlab = expression(lambda[2]),
98       col = "lightblue", border = "white", freq = FALSE)
99 lines(kde_lambda2, col = "red", lwd = 2)
100 abline(v = map_lambda2, col = "blue", lwd = 2, lty = 2)
101 abline(v = map_lambda2_kde, col = "#1e9600", lwd = 2, lty = 3)

```

```

99 abline(v = lambda2_summary$ci[1], col = "purple", lwd = 2, lty = 4)
100 abline(v = lambda2_summary$ci[2], col = "purple", lwd = 2, lty = 4)
101 legend("topright", legend = c("MAP KDE", "MAP samples", "95% CI (HPD)", col = c("#1e9600", "
    blue", "purple"), lwd = 2, lty = c(3, 2, 4), cex = 0.8)
102
103 hist(m_samples, breaks = 30, main = "m", xlab = "m", col = "lightblue", border = "white", freq =
    FALSE)
104 abline(v = map_m, col = "blue", lwd = 2, lty = 2)
105 abline(v = m_summary$ci[1], col = "purple", lwd = 2, lty = 4)
106 abline(v = m_summary$ci[2], col = "purple", lwd = 2, lty = 4)
107 legend("topright", legend = c("MAP samples", "95% CI (HPD)", col = c("blue", "purple"), lwd =
    2, lty = c(2, 4), cex = 0.8)
108
109 par(mfrow = c(1, 1))

```

The final numerical summaries are reported below, and the MAP estimates along with the HPD intervals are visualized in Figure 4.

	Mean	Median	SD	95% CI (HPD)	MAP (Samples)	MAP (KDE)
λ_1	3.21	3.20	0.23	[2.76, 3.67]	3.15	3.21
λ_2	5.11	5.10	0.32	[4.49, 5.75]	5.09	5.12
m	66.25	65	5.13	[59, 76]	65	-

Table 1: Posterior summaries for the model parameters.

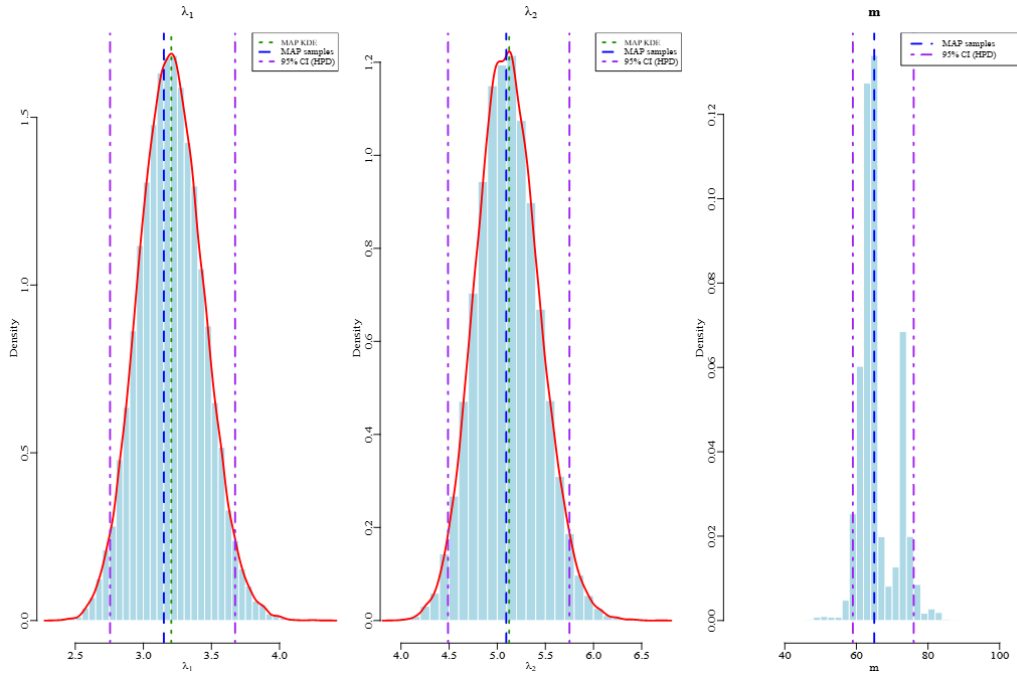


Figure 4: Histograms of the posterior samples for λ_1 , λ_2 , and m , along with their respective statistical summaries. The red curves are the KDEs.

2.6

Estimate the posterior probability $P(\lambda_2 > \lambda_1 | D)$. Interpret the result.

Analytically, this probability is calculated by integrating the joint posterior distribution over the region where $\lambda_2 > \lambda_1$. Using Monte Carlo methods, we can approximate this exact integral

by calculating the proportion of MCMC samples where the condition is true:

$$P(\lambda_2 > \lambda_1 | D) = \sum_{m \in \mathcal{M}} \int_0^\infty \int_{\lambda_1}^\infty p(\lambda_1, \lambda_2, m | D) d\lambda_2 d\lambda_1$$

$$\simeq \frac{1}{N_s} \sum_{i=1}^{N_s} I(\lambda_2^{(i)} > \lambda_1^{(i)}) \quad (8)$$

where N_s is the number of post burn-in MCMC samples, $I(\cdot)$ is the indicator function (which evaluates to 1 if the condition is true and 0 otherwise), and i indexes the paired samples drawn at the exact same Gibbs iteration.

The result with our chain is:

$$P(\lambda_2 > \lambda_1 | D) \simeq 1 \quad (9)$$

This implies almost certainly that the event rate increased after the change point. This result is entirely expected from the previous section, as the marginal posterior distribution of λ_2 is shifted entirely to the right of λ_1 , with almost zero overlap between their values.

```
1 # 2.f Estimate the posterior probability P(lambda2 > lambda1 | D)
2 # approx of double integral over the joint posterior of lambda1 and lambda2 where lambda2 >
  lambda1 as integration domain
3 prob_lambda2_greater <- mean(lambda2_samples > lambda1_samples)
4 print(paste("P(lambda2 > lambda1 | D) =", round(prob_lambda2_greater, 4)))
```

```
[1] "P(lambda2 > lambda1 | D) = 1"
```

3 Posterior predictive distribution

Let x_{new} be the number of events observed in the next time interval, after the end of the dataset.

3.1

Write the posterior predictive distribution $p(x_{\text{new}} | D)$ as an integral and sum over the posterior distribution of the unknown parameters. Explain why, for a future observation after time N , the relevant rate is λ_2 .

The posterior predictive distribution is obtained by marginalizing over all the parameters. This is done by taking the product of the conditional likelihood of a new observation x_{new} and the joint posterior distribution, and then integrating over the rate parameters and summing over all the possible change points $m \in \mathcal{M}$:

$$p(x_{\text{new}} | D) = \sum_{m \in \mathcal{M}} \int_0^\infty \int_0^\infty p(x_{\text{new}} | \lambda_1, \lambda_2, m) p(\lambda_1, \lambda_2, m | D) d\lambda_1 d\lambda_2 \quad (10)$$

Since the new observation x_{new} occurs at time step $N+1$, it falls after the change point ($N+1 > m$). Therefore, the generation of this future observation does not depend on λ_1 nor the exact position of m . Its conditional likelihood reduces to a Poisson distribution with rate λ_2 :

$$p(x_{\text{new}} | \lambda_1, \lambda_2, m) = p(x_{\text{new}} | \lambda_2) = \text{Poisson}(x_{\text{new}} | \lambda_2) \quad (11)$$

Substituting this back into the integral, we can sum the joint posterior distribution over m and integrate over λ_1 . Therefore the posterior predictive distribution depends only on the marginal posterior distribution of λ_2 :

$$p(x_{\text{new}} | D) = \int_0^\infty p(x_{\text{new}} | \lambda_2) \left[\sum_{m \in \mathcal{M}} \int_0^\infty p(\lambda_1, \lambda_2, m | D) d\lambda_1 \right] d\lambda_2$$

$$= \int_0^\infty p(x_{\text{new}} | \lambda_2) p(\lambda_2 | D) d\lambda_2 \quad (12)$$

3.2

Using your Gibbs sampler output, estimate the posterior predictive mean $E[x_{\text{new}}|D]$.

The posterior predictive mean can be evaluated by writing the expectation as an integral over the marginal posterior distribution of λ_2 :

$$E[x_{\text{new}}|D] = \int_0^\infty E[x_{\text{new}}|\lambda_2]p(\lambda_2|D) d\lambda_2 \quad (13)$$

Since x_{new} conditionally follows a Poisson distribution with rate λ_2 , its expected value is simply $E[x_{\text{new}}|\lambda_2] = \lambda_2$. Substituting this into the integral yields:

$$E[x_{\text{new}}|D] = \int_0^\infty \lambda_2 p(\lambda_2|D) d\lambda_2 = E[\lambda_2|D] \quad (14)$$

Therefore, we can estimate this mean directly by calculating the sample mean of our post burn-in MCMC draws for λ_2 :

$$E[x_{\text{new}}|D] \simeq \frac{1}{N_s} \sum_{i=1}^{N_s} \lambda_2^{(i)} \quad (15)$$

As calculated in the posterior summaries from Section 2.5, this estimated expected value is 5.11.

```
1 # 3.b Estimate the posterior predictive mean
2 print(paste("Posterior predictive mean:", round(mean(lambda2_samples), 2)))
```

```
[1] "Posterior predictive mean: 5.11"
```

3.3

Using your Gibbs sampler output, estimate $P(x_{\text{new}} > 2\bar{x}|D)$, where $\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$.

We can approximate the posterior predictive distribution by simulating new observations directly from our MCMC output. To minimize the variance, for every post burn-in sample $\lambda_2^{(i)}$ (where $i = 1, \dots, N_s$), we simulate K new independent observations $x_{\text{new}}^{(k,i)}$:

$$x_{\text{new}}^{(k,i)} \sim \text{Poisson}(\lambda_2^{(i)}) \quad \text{for } i = 1, \dots, N_s \text{ and } k = 1, \dots, K \quad (16)$$

In our implementation, we set $K = 1000$. This collection yields a total of $\tilde{N}_s = K \cdot N_s$ empirical samples representing the posterior predictive distribution $p(x_{\text{new}}|D)$.

To estimate the probability $P(x_{\text{new}} > 2\bar{x}|D)$, we apply Monte Carlo integration by calculating the proportion of these values that exceeds the threshold $2\bar{x}$:

$$P(x_{\text{new}} > 2\bar{x}|D) \simeq \frac{1}{\tilde{N}_s} \sum_{i=1}^{N_s} \sum_{k=1}^K I(x_{\text{new}}^{(k,i)} > 2\bar{x}) \quad (17)$$

where $I(\cdot)$ is the indicator function.

```
1 # 3.c Estimate the probability that the next count is greater than twice the mean of the
   observed counts
2 n_predictive_samples <- 1000 * length(lambda2_samples)
3 sample_predictive_distribution <- rpois(n = n_predictive_samples, lambda = lambda2_samples)
4 sample_mean <- mean(data$x)
5 prob_twice_mean <- mean(sample_predictive_distribution > 2*sample_mean)
6 print(paste("Probability that the next count is greater than twice the mean of the observed
   counts:", round(prob_twice_mean, 4)))
```

```
[1] "Probability that the next count is greater than twice the mean of
the observed counts: 0.0776"
```

3.4

Produce a histogram of simulated values from the posterior predictive distribution and mark $\bar{x}$ and $2\bar{x}$ on the plot.

Using the samples generated in the previous step, we can construct a histogram to visualize the posterior predictive distribution for the next observation.

```

1 # 3.d Histogram of posterior predictive distribution, with sample_mean and 2*sample_mean
2 par(mfrow = c(1, 1))
3 breaks = seq(min(sample_predictive_distribution) - 0.5,
4             max(sample_predictive_distribution) + 0.5,
5             by = 1) # ensure that each integer count gets its own bin
6 hist(sample_predictive_distribution, breaks = breaks, main = "Posterior Predictive Distribution"
7     , xlab = "Predicted Count", col = "lightblue", border = "white", freq = FALSE)
8 abline(v = sample_mean, col = "red", lwd = 2, lty = 2)
9 abline(v = 2*sample_mean, col = "green", lwd = 2, lty = 2)
10 legend("topright", legend = c("Mean of observed counts", "Twice the mean of observed counts"),
11     col = c("red", "green"), lwd = 2, lty = 2, cex = 0.8)
12 par(mfrow = c(1, 1))

```

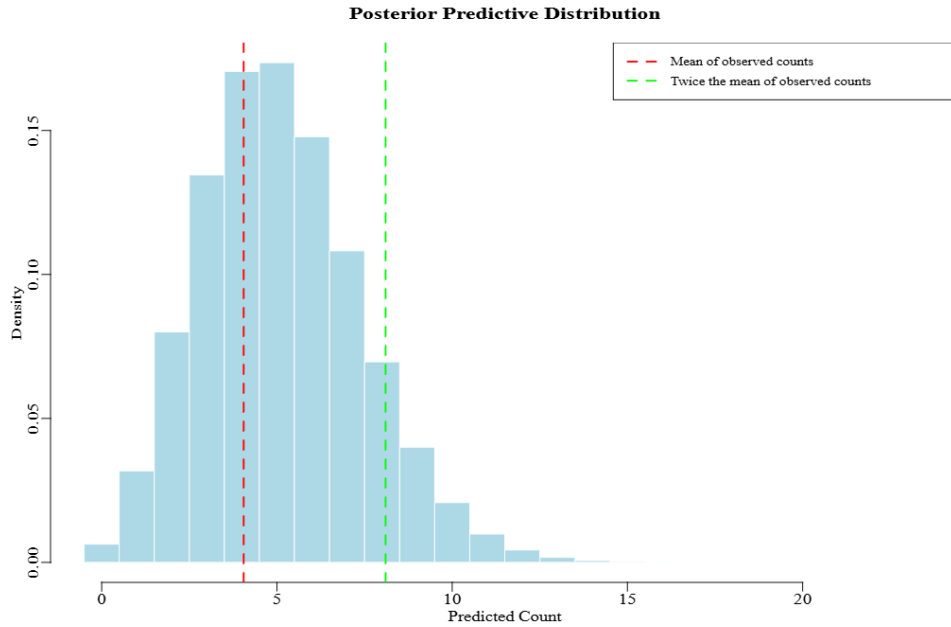


Figure 5: Histogram of the posterior predictive distribution for the next observation x_{new} .

4 Model comparison

Compare the one-change-point model M_1 with the simpler no-change model M_0 :

$$x_i | \lambda \sim \text{Poisson}(\lambda), \quad i = 1, \dots, N$$

with the same prior $\lambda \sim \text{Gamma}(\alpha, \beta)$.

4.1

Derive the marginal likelihood $p(D|M_0)$. You may ignore factors that are common to both models when computing the Bayes factor.

The marginal likelihood for model M_0 is obtained by integrating the product of the likelihood

function and the prior distribution over the rate parameter λ :

$$\begin{aligned}
p(D|M_0) &= \int_0^\infty p(D|\lambda, M_0) p(\lambda) d\lambda \\
&= \int_0^\infty \left(\prod_{i=1}^N \frac{\lambda^{x_i} e^{-\lambda}}{x_i!} \right) \left(\frac{\beta^\alpha}{\Gamma(\alpha)} \lambda^{\alpha-1} e^{-\beta\lambda} \right) d\lambda \\
&= \left(\prod_{i=1}^N \frac{1}{x_i!} \right) \frac{\beta^\alpha}{\Gamma(\alpha)} \int_0^\infty \lambda^{\alpha+S_N-1} e^{-(\beta+N)\lambda} d\lambda \\
&= \left(\prod_{i=1}^N \frac{1}{x_i!} \right) \frac{\beta^\alpha}{\Gamma(\alpha)} \frac{\Gamma(\alpha + S_N)}{(\beta + N)^{\alpha+S_N}}
\end{aligned} \tag{18}$$

where we recognized the integral of an unnormalized Gamma function. As suggested, when computing the evidence in R, we ignore the constant term $\prod_{i=1}^N \frac{1}{x_i!}$ because it represents a common factor that will appear identically in the marginal likelihood of both M_0 and M_1 and will thus cancel out in the Bayes factor ratio.

```

1 # 4. Model comparison
2 N <- length(data$x)
3 sumN <- sum(data$x)
4 Mset <- seq(5, N-5, 1)
5 len_M <- length(Mset)
6
7 # 4.a Marginal likelihood for M0 (on log scale, ignoring the 1/prod(x_i!) term)
8 log_marg_lik_M0 <- alpha_prior * log(beta_prior) -
9                   lgamma(alpha_prior) +
10                   lgamma(alpha_prior + sumN) -
11                   (alpha_prior + sumN) * log(beta_prior + N)
12 print(log_marg_lik_M0)

```

[1] 190.2929

4.2

Derive the marginal likelihood $p(D|M_1)$ by summing over $m \in \mathcal{M}$ and integrating over λ_1 and λ_2 .

To obtain the marginal likelihood for model M_1 , we must integrate the joint probability distribution over the continuous rate parameters (λ_1, λ_2) and sum over the discrete change point (m). We include all normalizing constants from the priors, including the uniform probability for the change point, $p(m) = \frac{1}{|\mathcal{M}|}$, where $|\mathcal{M}| = N - 6$ is the number of possible change points.

Because the likelihood factors into two independent Poisson processes conditioned on m , we can decouple the double integral into the product of two separate integrals:

$$\begin{aligned}
p(D|M_1) &= \sum_{m \in \mathcal{M}} \int_0^\infty \int_0^\infty p(D|\lambda_1, \lambda_2, m) p(\lambda_1) p(\lambda_2) p(m) d\lambda_1 d\lambda_2 \\
&= \frac{1}{|\mathcal{M}|} \left(\prod_{i=1}^N \frac{1}{x_i!} \right) \sum_{m \in \mathcal{M}} \left[\int_0^\infty \lambda_1^{S_m} e^{-m\lambda_1} p(\lambda_1) d\lambda_1 \right] \left[\int_0^\infty \lambda_2^{S_N-S_m} e^{-(N-m)\lambda_2} p(\lambda_2) d\lambda_2 \right]
\end{aligned} \tag{19}$$

Next, we evaluate the two integral terms recognizing the unnormalized Gamma functions, as done with model M_0 . For λ_1 , the integral is:

$$\begin{aligned}
\int_0^\infty \lambda_1^{S_m} e^{-m\lambda_1} \frac{\beta^\alpha}{\Gamma(\alpha)} \lambda_1^{\alpha-1} e^{-\beta\lambda_1} d\lambda_1 &= \frac{\beta^\alpha}{\Gamma(\alpha)} \int_0^\infty \lambda_1^{\alpha+S_m-1} e^{-(\beta+m)\lambda_1} d\lambda_1 \\
&= \frac{\beta^\alpha}{\Gamma(\alpha)} \frac{\Gamma(\alpha + S_m)}{(\beta + m)^{\alpha+S_m}}
\end{aligned} \tag{20}$$

Similarly, for λ_2 :

$$\begin{aligned} \int_0^\infty \lambda_2^{S_N - S_m} e^{-(N-m)\lambda_2} \frac{\beta^\alpha}{\Gamma(\alpha)} \lambda_2^{\alpha-1} e^{-\beta\lambda_2} d\lambda_2 &= \frac{\beta^\alpha}{\Gamma(\alpha)} \int_0^\infty \lambda_2^{\alpha+S_N-S_m-1} e^{-(\beta+N-m)\lambda_2} d\lambda_2 \\ &= \frac{\beta^\alpha}{\Gamma(\alpha)} \frac{\Gamma(\alpha+S_N-S_m)}{(\beta+N-m)^{\alpha+S_N-S_m}} \end{aligned} \quad (21)$$

Finally, multiplying these two results and placing them back into the summation, we can rewrite the exact marginal likelihood as:

$$p(D|M_1) = \left(\prod_{i=1}^N \frac{1}{x_i!} \right) \frac{\beta^{2\alpha}}{|\mathcal{M}|\Gamma(\alpha)^2} \sum_{m \in \mathcal{M}} \frac{\Gamma(\alpha+S_m)}{(\beta+m)^{\alpha+S_m}} \frac{\Gamma(\alpha+S_N-S_m)}{(\beta+N-m)^{\alpha+S_N-S_m}} \quad (22)$$

Also here we neglect $\prod_{i=1}^N \frac{1}{x_i!}$ term as it cancels out when computing the Bayes factor.

```

1 # 4.b Marginal likelihood for M1 (on log scale, ignoring the 1/prod(x_i!) term)
2 log_terms_M1 <- numeric(len_M)
3
4 for (i in 1:len_M) {
5   m <- Mset[i]
6   sumM <- sum(data$x[1:m])
7   log_terms_M1[i] <- 2 * alpha_prior * log(beta_prior) -
8     log(len_M) -
9     2 * lgamma(alpha_prior) +
10    lgamma(alpha_prior + sumM) +
11    lgamma(alpha_prior + sumN - sumM) -
12    (alpha_prior + sumM) * log(beta_prior + m) -
13    (alpha_prior + sumN - sumM) * log(beta_prior + N - m)
14 }
15
16 log_marg_lik_M1 <- log(sum(exp(log_terms_M1)))
17 print(log_marg_lik_M1)

```

```
[1] 198.5858
```

4.3

Compute the Bayes factor $BF_{10} = \frac{p(D|M_1)}{p(D|M_0)}$ and interpret the result. Which model is favored, and to which degree?

We can compute the log Bayes factor as the difference between the log marginal likelihoods of the two models:

$$\log(BF_{10}) = \log(p(D|M_1)) - \log(p(D|M_0)) \simeq 8.29 \quad (23)$$

where the final value is obtained by substituting the values of our specific dataset statistics and prior hyperparameters. Hence, we can state that the data provides decisive evidence in favor of model M_1 over the M_0 model.

```

1 # 4.c Compute the log Bayes factor
2 log_BF10 <- log_marg_lik_M1 - log_marg_lik_M0
3 sprintf("Log Bayes Factor (log BF10): %.4f", log_BF10)

```

```
[1] "Log Bayes Factor (log BF10): 8.2929"
```

5 Discussion

Shortly summarize your conclusions. Your discussion should address in a precise but concise way the following points:

- *Is there evidence for a change point in the event rate?*
As calculated in the model comparison section, the log Bayes factor provides decisive statistical evidence in favor of the change point model M_1 over the constant rate model M_0 . We can confidently conclude that a change in the event rate occurred.
- *What is the most plausible location of the change point?*
The most plausible location is given by the MAP estimate, although the evident bimodality of the posterior distribution indicates some ambiguity regarding the exact location of the change point. According to our posterior samples, the change point most likely occurred at $m = 65$, with a 95% HPD credible interval of $[59, 76]$.
- *Are the two rates clearly separated according to the posterior distribution?*
The two rates are entirely separated, as the 95% HPD credible intervals for the two rates ($\lambda_1 \in [2.76, 3.67]$ and $\lambda_2 \in [4.49, 5.75]$) are mutually exclusive with no overlap. Furthermore, as demonstrated previously, the posterior probability $P(\lambda_2 > \lambda_1 | D)$ is effectively equal to 1.
- *Did the different chains give consistent results?*
The chains yielded consistent results. Indeed, visual inspection of the trace plots confirms that, despite starting from different initial values, all three MC rapidly converged to the same stationary distribution and explored the same typical set.
- *How large are the autocorrelations? How reliable are the effective sample sizes?*
The autocorrelations are small, rapidly decaying to zero after approximately 5 lags. Because the chains mix efficiently, the resulting Effective Sample Sizes are highly reliable and large, retaining approximately 77%, 83%, and 72% of the post burn-in samples for λ_1 , λ_2 , and m , respectively.
- *The model contains a discrete parameter m . Explain why Hamiltonian Monte Carlo cannot be applied directly to the full state $(\lambda_1, \lambda_2, m)$ without modifying the model or marginalizing over m .*
Hamiltonian Monte Carlo (HMC) relies on simulating Hamiltonian dynamics, which requires the target log-posterior density to be continuous and differentiable with respect to all its parameters. Because m is a discrete parameter, its state space is not continuous and gradients cannot be evaluated. Therefore, HMC cannot be directly applied to the joint posterior distribution unless m is first marginalized out.